

Sleeping Machines: what is known

Computing in time with races, holds and vetoes · report, 27 September 2026

Sleeping Machines proposes that computation can happen **in time rather than memory**: candidate events race, the first to fire cancels the rest, and what a node computes is set by delays, by how long it holds an input, and by inhibition that arrives in time. This report states what is now known about such networks, why, and what remains open. Derivations and proofs are in experiments/THEORY.md (cited as §n).

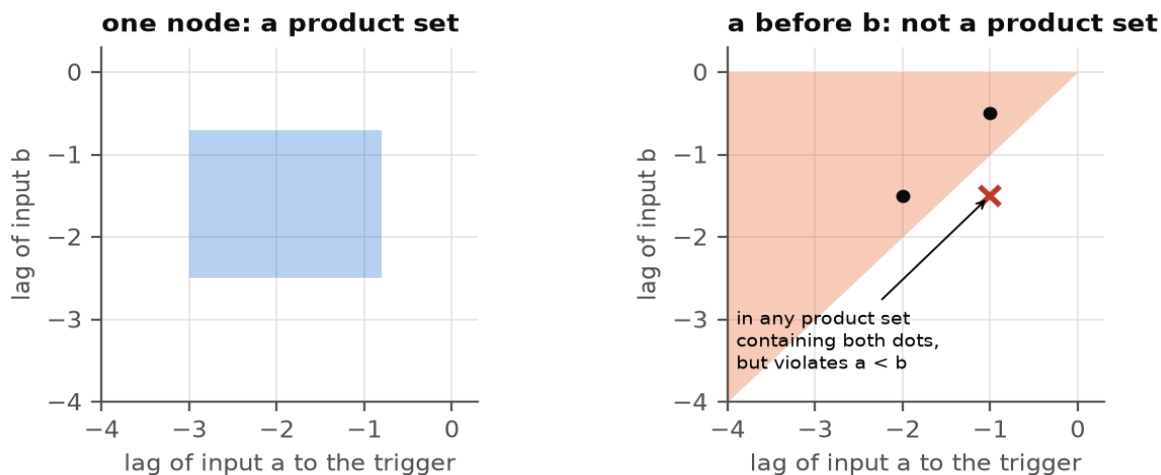
Summary

- **What one node computes is exactly characterized, and it says where depth is needed.** A node with a trigger, hold and veto inputs accepts a product set in lag coordinates; one node orders at most three events; two layers compute every conjunction of bounded time differences, three layers every union (§71, proved; exhaustive search agrees). Nodes with state (arm/disarm) are strictly stronger.
- **Event networks learn timing with very few mistakes** if winning is positional (pull the event that should have won, never push a loser later) and order uses held intervals, not aligned delays: $O(\log N + \text{a few})$ mistakes per node.
- **On a timing task a learned event network matches or beats dense models at 10^4 - $10^5\times$ lower cost**, with nothing given: 1.000 at 7.5 synaptic events per episode vs 0.989–0.996 for event-token Transformers at 146k–1.16M multiply-adds and 0.995 for a clocked conv net at 3.07M.
- **Depth pays when composition is a hold/trigger chain** (0.97 vs 0.39 at depth 1), but a well-trained Transformer is more accurate on the composition task (0.998) at $\approx 10^4\times$ the cost.
- **Grokking occurs, by a route change under sleep, and only for relations the substrate can express.** Without sleep the network memorizes; with sleep it generalizes after a delay (0.93–0.99, reliable with cooled timing noise); a data $\times$ sleep phase diagram shows memorization, grokking and collapse; with depth, $(a + b + c) \bmod p$ through two composed rhythm stages reaches 0.998 (pilot).
- **Learning cost follows activity, not model size:** 12 $\rightarrow$ 48 input channels leaves learning mistakes flat and makes inference cheaper (§77).
- **Not yet: real asynchronous benchmarks.** SHD below dense baselines; on the market, correctly posed as trading with costs, no learner profits and the native one learns to stay out; an online world model of the stream matches a Hawkes process but not a neural point process.

1. What an event node computes

Primitives. Spike times; delay; first-of (min) and all-of (max); hold (an input opens a window of given duration); veto (an input blocks the node while within its window). A node fires at its trigger if its holds are met and no veto is active.

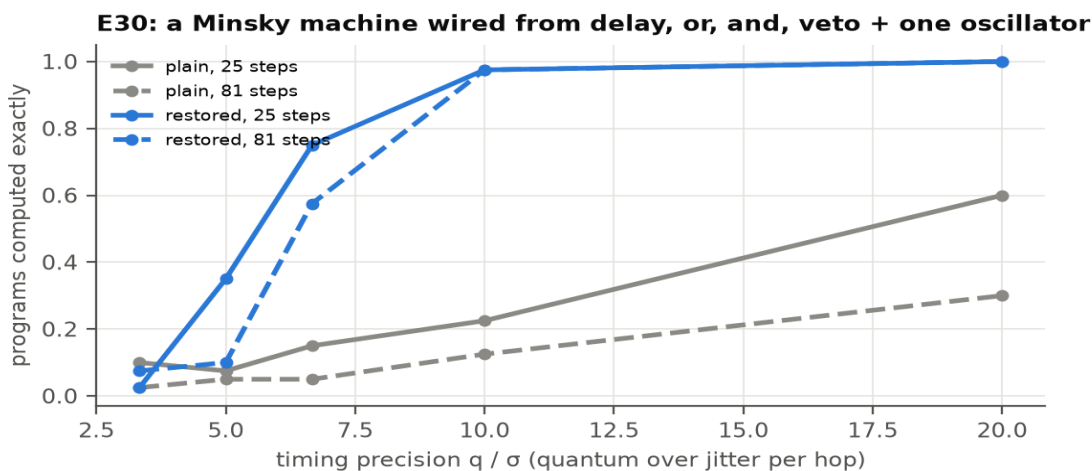
Theorem (§71). In lag coordinates relative to the trigger a stateless node accepts a product set. So one node orders at most three events (trigger the middle one, hold the first, veto the last until it arrives); two layers compute every conjunction of bounded differences with exclusions (every zone of timed-automata verification); three layers every finite union. *Evidence:* an exhaustive search over single nodes finds the proof's own node for 3 events and none for 4 on grids of 1,680 and 11,880 configurations; all thirteen Allen interval relations built at depth 2 are exact on ~19,900 interval pairs (E39).



Stateful nodes are stronger. A node armed by A, disarmed by C and fired by B computes an XNOR of two order relations, which no stateless node computes (exhaustive search, E39b); a toggle node computes parity.

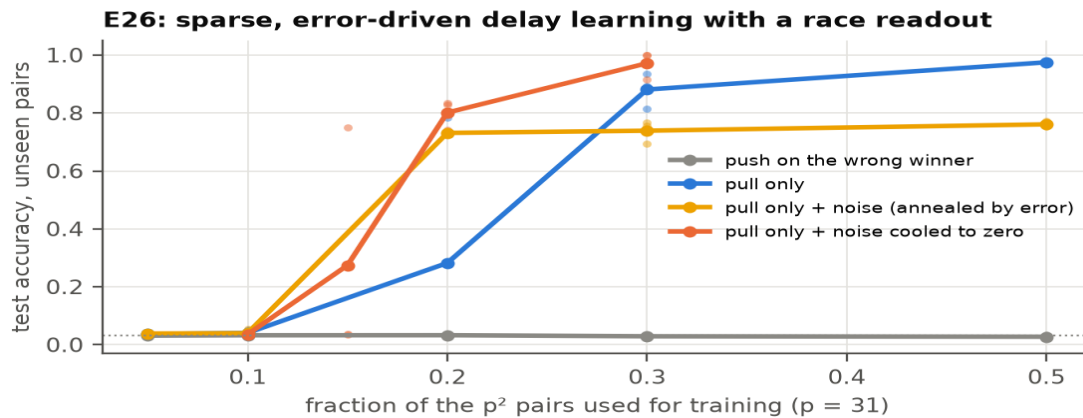
Clockless computation computes relations, not sums: shift symmetry forbids adding two times; one shared rhythm makes cyclic arithmetic computable (§56).

Completeness and reliability (E30). A two-counter machine wired from these primitives plus one oscillator runs exactly (Turing-complete given timing precision). Jitter accumulates like a random walk; one restoring coincidence per cycle makes reliability independent of program length.



2. How event networks learn

Winning is positional (§54). Pull the event that should have won; never push a loser later, which hands the win to the next loser and collapses the detectors. With learned delays on $(a + b) \bmod p$: push at chance in all 15 runs, pull-only 0.97–0.98 (E26). Timing noise cooled to zero makes small-data generalization reliable.



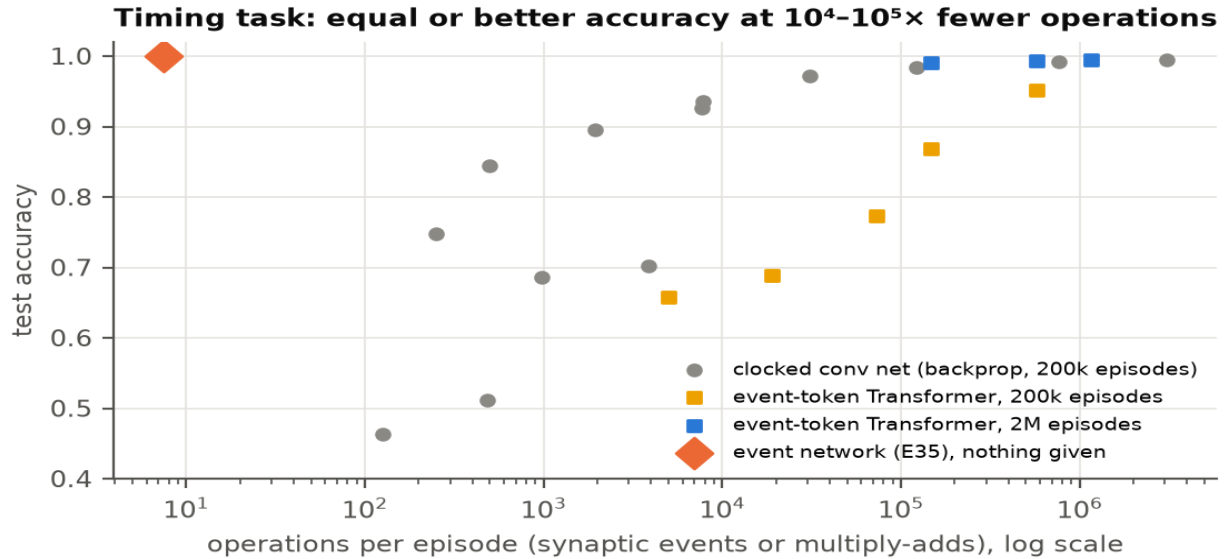
Order needs held intervals, not aligned delays (§61). Delaying A to meet B compresses the interval so a veto cannot see where C fell; A opening a window that B must hit keeps it. With holding, the detector for “B within Δ after A unless C” reaches 1.000 on 5 seeds with ~450 updates in 200k episodes, and veto is worth 19 points (alignment: 0.914, veto worth 1.8).

The rules are online learning on the simplex (§68): pulls moving a fixed fraction of a conserved budget are Winnow/Hedge-type updates with $O(k \log N)$ mistake bounds; durations are interval learning, vetoes monotone disjunctions. With nothing given, detectors learn channels, durations and vetoes: 1.000×4 , 0.9995, 443–1,530 updates (E35).

Routing needs counterfactuals (§57). Credit along a spike's causal path cannot say whether another route should have been taken; cancelled near misses supply it. Without it a two-layer network stays at chance (0.17 vs 0.72–0.74, E28).

3. Against dense models and Transformers

model	test accuracy	cost per episode	learning
event network, nothing given (E35)	1.000 ×4, 0.9995	7.5 synaptic events	443–1,530 updates, 200k episodes
event-token Transformer, 2M episodes	0.989–0.996	146k–1.16M MACs	backprop
event-token Transformer, 200k episodes	0.65–0.97	5k–576k MACs	backprop
clocked conv net (E32)	0.995 / 0.984 / 0.895	3.07M / 123k / 1.9k MACs	backprop, 200k

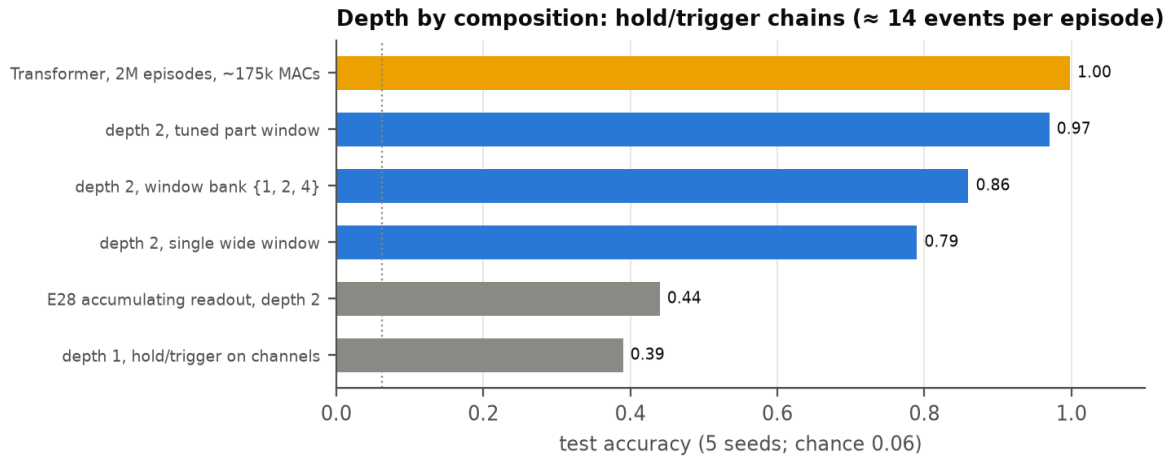


Scaling with the input basis (E35, §77). At fixed spikes per episode, $12 \rightarrow 48$ candidate channels leaves accuracy ≈ 1.0 and learning updates flat while synaptic events per episode fall: learning and inference cost follow activity, not the size of the basis.

Why. Attention cannot see order without position information and must synthesize time comparisons from dot products at $O(n^2 \cdot d)$ per layer; a hold/trigger node computes the comparison as its primitive, at the cost of its input events (§70). The clocked model also pays per time bin, so silence multiplies its cost; a conv net evaluated only where spikes are would cost ≈ 100 multiply-adds, not millions (§63). At published per-operation energies the gap at equal accuracy is $\approx 10^4\times$. *Limits:* one task built around the primitives; the Transformer needed $10\times$ more training for parity; a relative-time-attention Transformer is being run.

4. Depth and composition

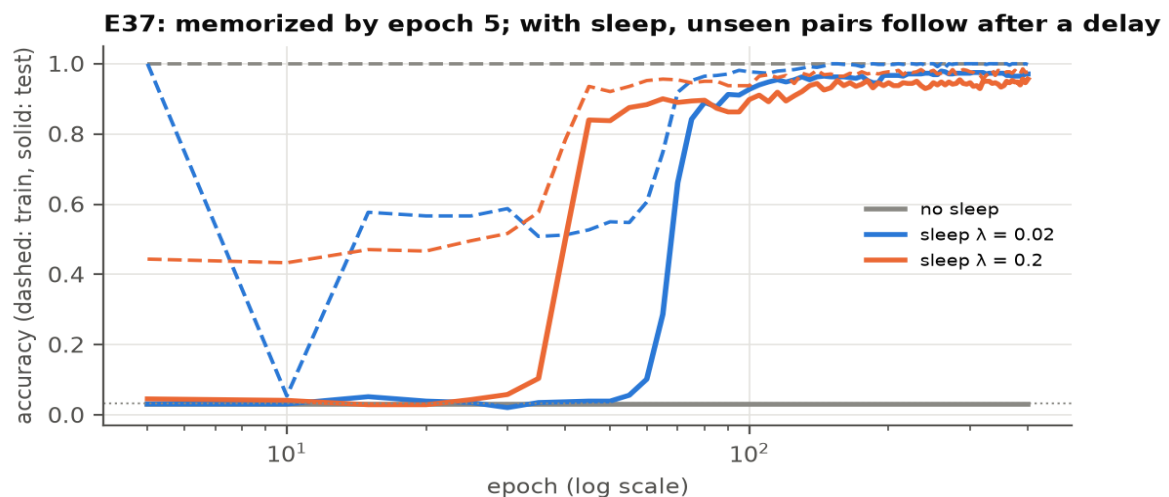
Classes that are ordered combinations of shared parts are zones over part events, so the theorem prescribes two layers: parts (hold nodes on channel pairs) and class nodes that hold one part's spike and are triggered by another's, so a single part cannot satisfy them. Learned natively: 0.97 with part windows at the motif scale, 0.86 with a generic bank of window scales, depth 1 0.39 (5 seeds, E34). A readout that accumulates evidence fails at conjunction even though its hidden nodes learn parts (E28). An event-token Transformer trained on 2M episodes reaches 0.998 at $\approx 175\text{k}-690\text{k}$ multiply-adds vs ≈ 14 events. The chains are not training-limited: at 200k episodes they stay at 0.954 (tuned) and 0.87 (bank); some class routes lock onto wrong parts.



5. Generalization and grokking

What counts (§58): restriction, forced generalization above capacity, and grokking (the relation reached while memorizers are available) are different claims; each reports $p = n/\text{params}$. **Per-class parameters cannot generalize on $(a + b) \bmod p$ (§66):** each operand occurs once per class, so generalization needs shared intermediates; in time, a sum needs a rhythm.

Grokking as a route change (E37). Each class has a pair-node lookup that can memorize everything ($p \approx 0.016$) and a shared route through a rhythm with learned delays; learning is errors-only. Without sleep: train 1.0, test 0.03–0.04. With sleep ($\lambda = 0.02-0.2$): test 0.93–0.97 in 2 of 3 seeds, after a delay; the third seed collapses. Sleep without the rhythm: train 0.45, test 0. The rhythm is used only where it fits: $a - b$ and relabelled sums grok (0.95–0.98), $a \cdot b$ in 1 of 3 seeds, while $a^2 + ab + b^2$ and random tables stay at chance on unseen pairs (0.01–0.04) and their training accuracy erodes under sleep. A data $\times$ sleep phase diagram (4×4 , 3 seeds) shows no grokking below 20–30% of pairs, and above it a minimum sleep that falls with data. Larger problems grok more reliably: 2/3 seeds at $p = 31$ and 59, 3/3 at $p = 97$ ($p \approx 0.005$). With depth (E41, pilot): $(a + b + c) \bmod p$ through two composed rhythm stages, 0.998 on unseen triples with sleep, 0.06 without.

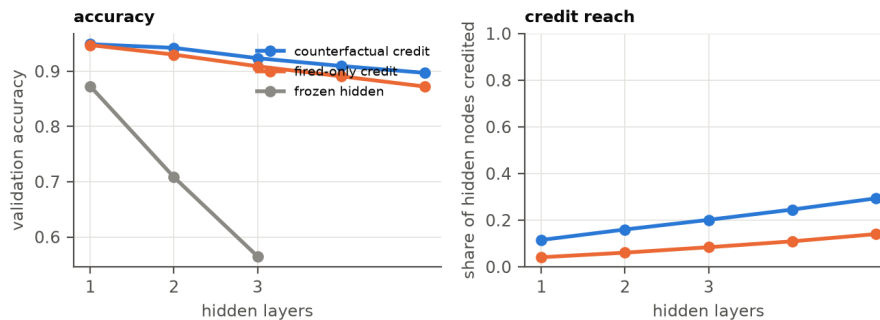


Why (§69, §72). Error-gated learning makes memorization absorbing. Sleep keeps a parameter only if it is used by more than $m^* = \lambda\theta/(e\eta)$ examples: lookup entries serve one and die, the rhythm's delays serve many and survive, and once the rhythm answers a pair its lookup entry is never relearned. This predicts memorization, grokking and collapse regimes (all observed) and a data threshold $n^* \propto p\lambda\theta/(e\eta)$.

6. The weight race

The original architecture (integrate-to-threshold nodes with learned weights, racing in groups) holds its first results: a cancelled node can be taught (E4: 0.79 at $K = 128$ where reward-modulated rules are at chance); races decide as fast as the evidence allows (E2, matching the optimal MSPRT); learning work tracks activity (E5: $\approx 300\times$ fewer updates); local learning reaches ≈ 0.96 on latency-coded MNIST with counterfactual credit that pays more with depth (E6, E14).

E14 · the advantage of counterfactual credit grows with depth (2 seeds)



Its limits: depth still costs accuracy; only its single racing layer beats an equal dense model at inference ($\approx 2.4\times$); it forgets more than SGD (E23); it memorizes instead of grokking (E24); and the rules that fixed the timing networks do not transfer to its weights (SHD 0.04–0.29 vs 0.35).

7. Real data

- **Spiking Heidelberg Digits:** the weight race 0.35 vs 0.56–0.59 for a dense MLP. For the timing architecture the representation is the bottleneck: onset-referenced parts give a dense readout 0.566, and the native learner overfits (0.27–0.33). SHD is only $\approx 6\times$ sparser than a 10 ms raster, a weak test of the paradigm.
- **Market stream posed as trading with costs (E42, pilot):** imitating a hindsight teacher over-trades and loses; a profit-priced event learner makes 26 changes in 7 days (–170 bp): it learns that trading does not pay.
- **Online world model (E44):** a temporal point process of four event types learned from every event; the native model (–2.64/–2.85/–2.70 nats per event) matches Hawkes (–2.62/–2.94/–2.84) and trails a GRU neural point process (–2.61/–2.52) by ≈ 0.2 nats.

8. Open problems and next steps

- Grokking theory tests (running): phase diagram over data $\times$ sleep (§72); relations the rhythm cannot express; sleep as the pressure toward reusable parts (§73); $O(\log N)$ learning as the candidate basis grows (§74).
- Fair baselines (running): relative-time-attention Transformers; the chains with 5–25 $\times$ more training.
- Composition accuracy against Transformers; grokking reliability; native learning of sparse parity (§75, open); a real stream with rare, precisely timed events (§55).

Every mechanism is an event handler (local state, triggered by events, cost proportional to events); dense procedures are diagnostics only. Reproduce: `python report/figures_time.py` && `python report/make_pdf.py`.